AI Engineer World's Fair

FROZEN ENCODER · TEST-TIME COMPUTE · EMERGENT TASK

Squeezing a new skill out of a frozen embedding model at test time

A 1B embedding model was never trained to tag images. With zero training and no second model, it does it anyway. How much of that is test-time compute?

GITHUB REPO

the code

Han Xiao
VP of AI, Elastic

@hxiao · in/hxiao87

WHERE WE LEFT OFF

Last talk: search is test-time compute over a frozen encoder.

The claim from the autoresearch talk: you do not need a bigger model to get better retrieval. You **recombine the geometry** of one frozen encoder at inference: split the document, re-score, fuse channels, feed back. That extra work at test time is the compute.

THE MODEL

frozen. One embedding API. No retraining, no new parameters.

THE COMPUTE

multi-pass embedding algebra assembled at test time, climbing a cost-versus-quality curve.

Same idea, new question: instead of squeezing more relevance out of the encoder, can we squeeze out an entirely new task?

THE MOVE

Point the same lens at a task the model was never trained to do.

jina-embeddings-v5-omni-nano is a ~1B retrieval model. It embeds text and images into one space for search. It has **no tagging head, no classifier, no tag list**. It was never trained to answer "what is in this image".

THE NAIVE PATH

train a tagging head. Curate a label set. Fine-tune. A second model, more data, more parameters.

THIS TALK

keep the model frozen. Manufacture the whole tagger out of test-time compute over its existing geometry.

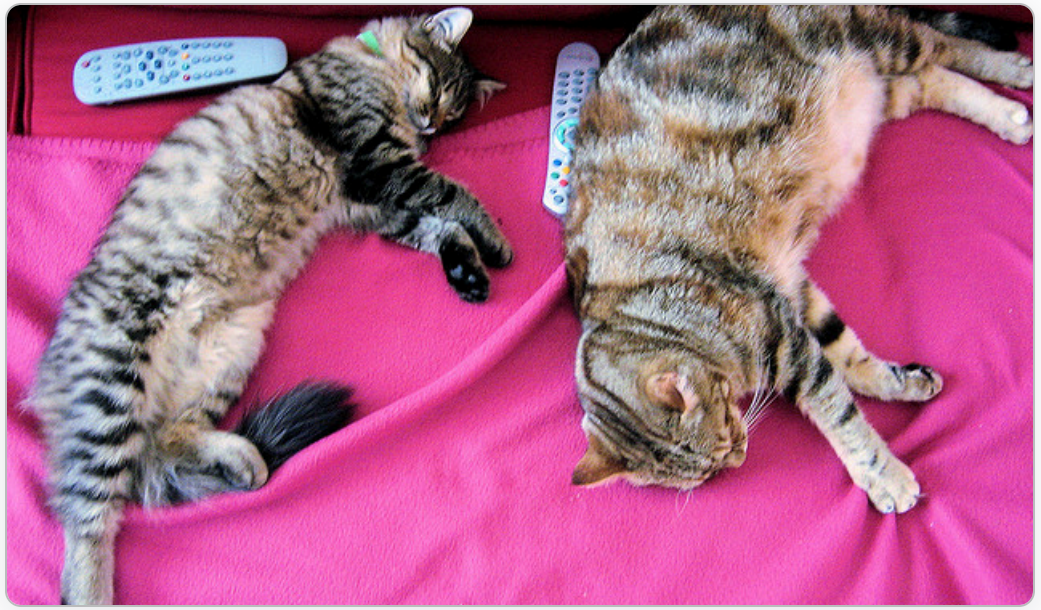
If a skill it was never trained on emerges purely from inference-time work, that is test-time compute buying capability, not just quality.

THE TASK

Open-vocabulary, multi-label image tagging.

Give it an image, get back the words for what is in it. Not one label: **every** object present. Not a fixed 80-class head: an **open vocabulary**, any word the model knows.

OPEN-VOCABULARY	no curated tag list; the label space is the model's own.
MULTI-LABEL	a photo has many objects; recall them all.
SINGLE FROZEN MODEL	no WordNet, no POS tagger, no second network.



cat.jpg (86ms) → kitty, cosy, plush, crib, paw, sleeps

THE RULES

Every signal comes from that one frozen model. Nothing else.

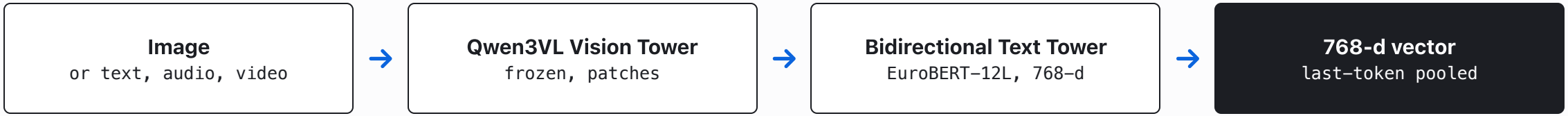
ZERO TRAINING | NO SECOND MODEL | NO WORDNET / DICTIONARY | NO REGEX / POS TAGGER

This is the same discipline as the retrieval talk, ported to a new task. The label space, the visual features, and the word filter must **all** come from the frozen encoder. The moment you add a curated tag list or a POS tagger, it is no longer the model doing the work.

The constraint is the point: it forces every ounce of the new skill to come from test-time compute over existing geometry.

THE MODEL

One frozen omni encoder, image and text in one space.



~1B
parameters, runs on a Mac in MLX

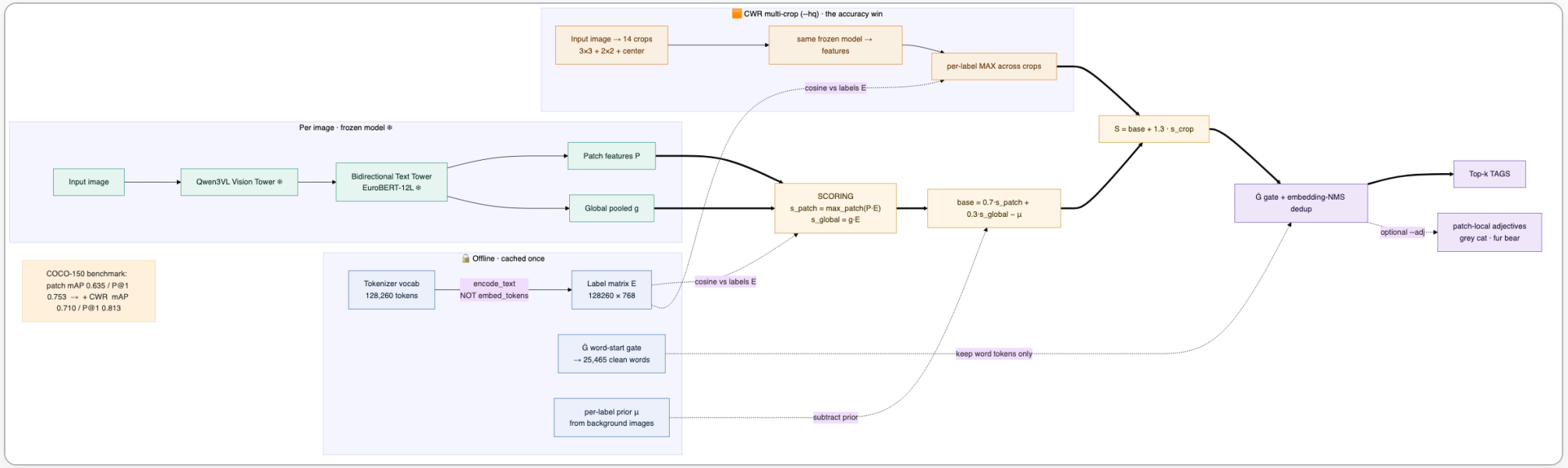
768
shared image-text dimensions

1
model for every modality and every signal

Because image and text land in the same space, an image can be scored directly against words. That shared geometry is the raw material.

THE PIPELINE

The whole tagger is test-time compute around a frozen forward pass.



Offline (once): encode the tokenizer vocabulary into a label matrix. Per image: score patches and the global vector against those labels, subtract a prior, optionally boost with multi-crop, then filter to words. **Every box is the same frozen model or plain arithmetic.**

STEP 1 · THE LABEL SPACE

The label set is the tokenizer's own vocabulary.

No curated tag list. Take all **128,260** tokens and encode each one `as text` into a label matrix. Now any image can be scored against the entire vocabulary at once.

```
wrong: image_vec · embed_tokens.weight
       → Tutor, avatar, PyTuple  (garbage)

right: image_vec · encode_text(token)
      → kitty, cosy, plush, paw  (aligned)
```

Why encode_text, not the embedding table? The model has `tie_word_embeddings = false`: the input-embedding table lives in a different space than the pooled output. You must encode each token *as a text string* to put labels in the image's space.

Han's original intuition ("just use the vocab") was right, but only through the aligned output space.

STEP 2 · WHERE TO LOOK

One vector per image is a weak tagger. Score every patch.

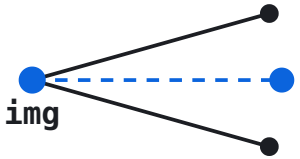
GLOBAL POOLED

$\cos(g, E)$

one vector dominated by the most salient object

MAP 0.264

PATCH MAX + GLOBAL

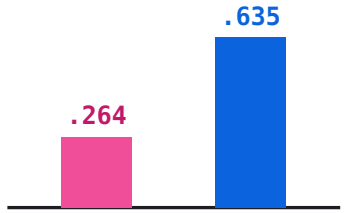


$0.7 \max_p \cos(P, E) + 0.3 \cos(g, E)$

a small object fires on its own patch and survives

MAP 0.635

THE GAIN



+0.37 mAP

"[CLS] is not enough", confirmed on this encoder

NO NEW PIXELS, JUST LOOK CLOSER

Scoring each patch and taking the per-label max is multi-pass compute over the same features: more work at test time, no new model.

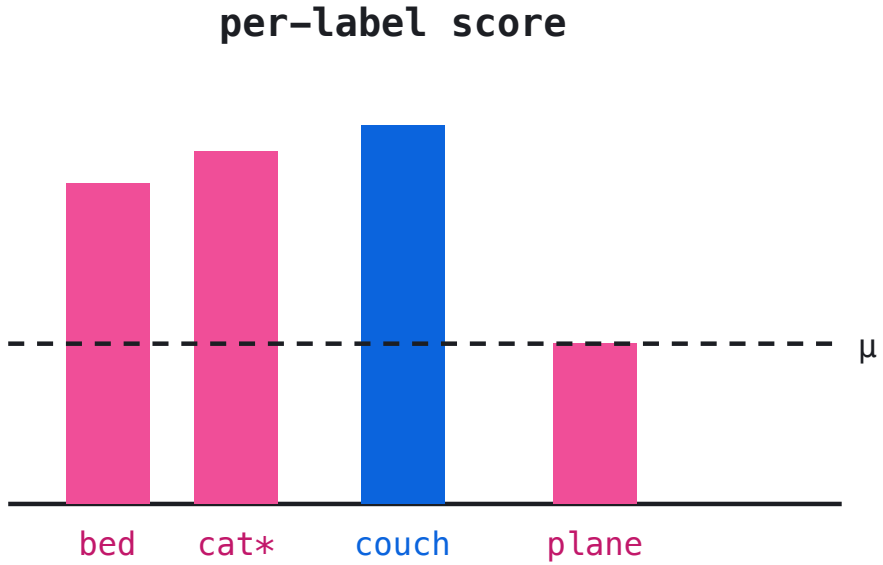
STEP 3 · DE-BIAS

Some words are just close to every image. Subtract that.

Raw cosine has a base-rate bias: generic words ("bed", "cat") score high on almost any image because they sit near the image modality. Estimate a per-label prior μ from a handful of background images once, then subtract it.

```
base = 0.7·s_patch + 0.3·s_global − μ
```

Pure marginal de-biasing: no calibration set, no labels, one offline pass over neutral images. It is the lightest possible correction, and it is enough because the space is already well aligned.



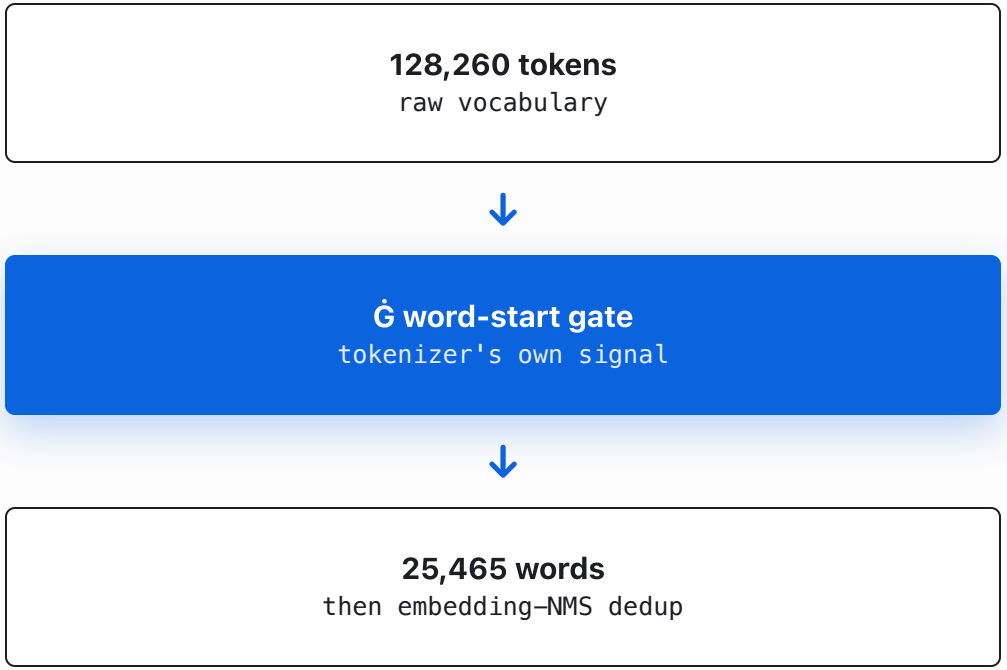
subtract μ: only true spikes survive

STEP 4 · CLEAN UP

The tokenizer already knows what a word is.

Word-start gate. Byte-BPE marks whole-word starts with a space glyph. `Ġcat` is a word; `GetComponent` is a fragment. That one built-in signal filters 128k tokens down to **25,465** clean words. No external dictionary.

Embedding-NMS. Synonyms and multilingual variants (/ Cat / кот / cat) collapse into one, using cosine in the model's own space. Again: no lookup table, just the geometry.



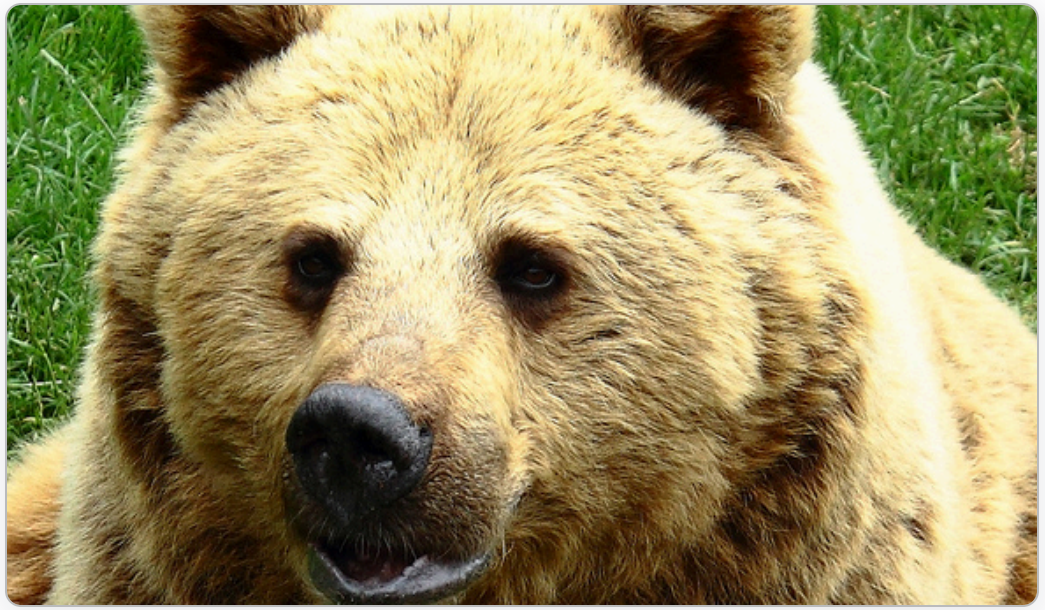
STEP 5 · THE ONE REAL LEVER

CWR multi-crop: the only move that genuinely lifts accuracy.

Re-encode the image as a **3×3 + 2×2 + center** grid of 14 crops through the same frozen model, and take the per-label **max** across crops. A small object becomes large in whichever crop contains it, so its signal jumps from weak to strong.

```
S = base + 1.3 · s_crop
```

This is test-time augmentation done right: full-coverage grid plus per-label max. Blind center-crop averaging hurt; the outlier crop is not noise, it is the evidence.



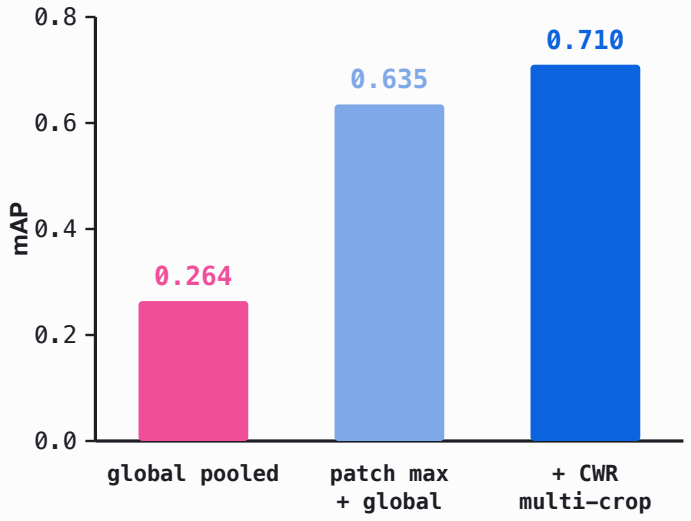
fast: wolf, roar, muzzle → --hq: fur, **bear**, muzzle
CWR recovers the small bear the single pass misses

RESULTS

COCO-150, 80 categories: a frozen retrieval model that tags.

METHOD	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

150 COCO-val images, real multi-label ground truth, avg 2.93 labels/image.



mAP: global → patch → +CWR

IN THE WILD

Open vocabulary, out-of-distribution photos.

AI conference hall

onstage · attendees · ceilings · seats

Porsche 911 interior · --hq

carro · leather · steering · clock · seat

bear on grass · --hq

fur · bear · muzzle · ivory

Labels come straight from the tokenizer, so multilingual variants ("carro") and the odd fragment appear. That is the honest price of a zero-annotation open vocabulary.

THE REAL EXPERIMENT

So where is the ceiling? Spend more test-time compute, keep winning?

The tagger works. The scientific question is the same one from the retrieval talk: **does accuracy scale with the compute you pour in at test time?** We systematically implemented the obvious upgrades from the 2024-2026 training-free literature and measured each on COCO-150.

RE-PROCESS THE FEATURES

change the layer, whiten, softmax over classes, propagate on a graph, optimal transport, robust aggregation.

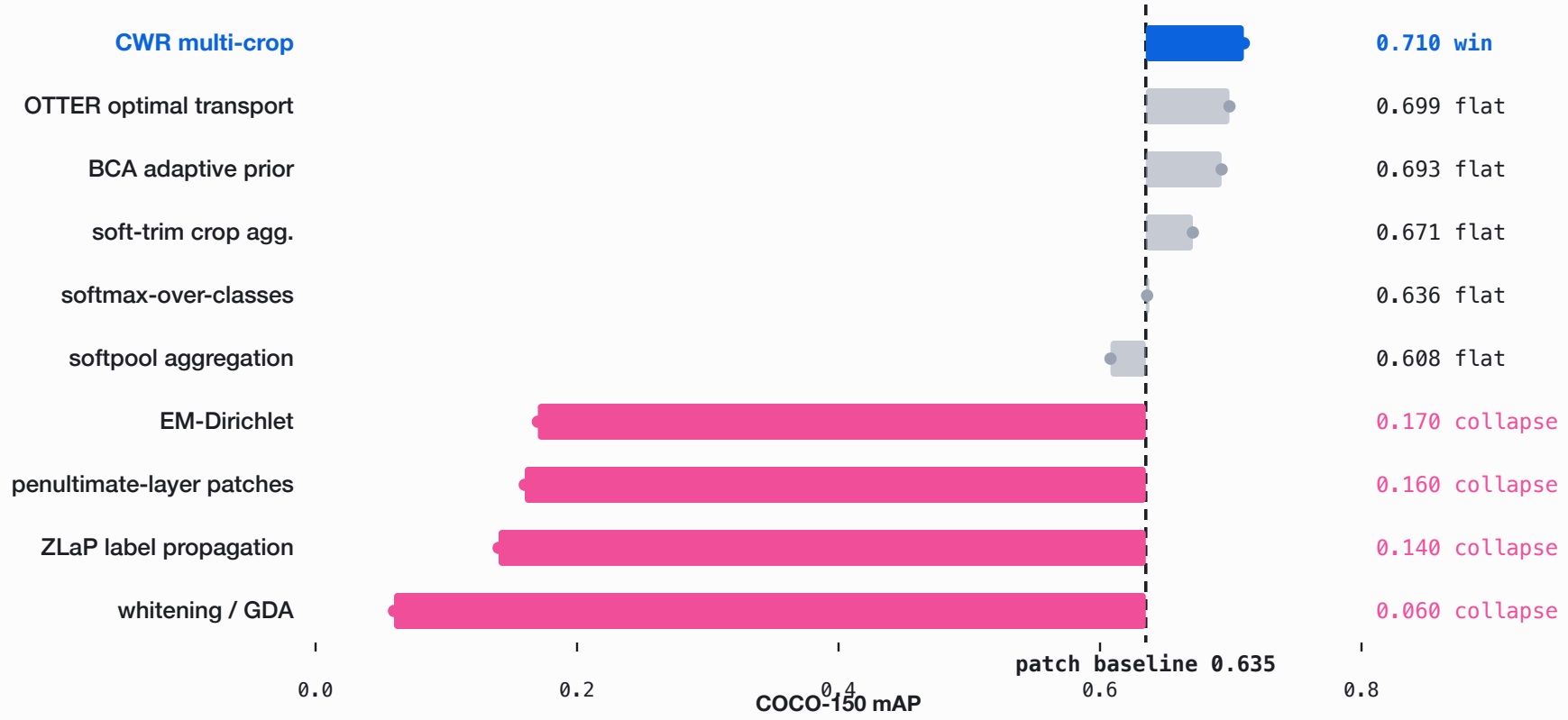
FEED THE MODEL NEW INFO

CWR: run the frozen encoder again on crops it has not seen at that scale.

One of these two families works. The other does nothing, or collapses.

EVERYTHING WE TRIED

Nine "advanced" test-time methods. Only one beats the baseline.



THE META-CONCLUSION

Re-processing good features does nothing. Feeding new pixels works.

Every method that **re-processes** the existing features - new layer, whitening, cross-class softmax, label propagation, optimal transport, Dirichlet, robust trimming - is a no-op or a collapse. They were designed for weakly calibrated single-label CLIP. This encoder's space is already good and already multi-label, so there is nothing left to recover.

RE-PROCESSING FEATURES

whitening 0.06 · ZLaP 0.14 · EM-Dirichlet 0.17 · softmax 0.636 · OTTER 0.699. All ≤ baseline.

FEEDING NEW INFORMATION

CWR crops → mAP 0.710, P@1 0.813. The only lever that moves the ceiling.

The ceiling is the 1B model itself. Real test-time compute here means giving the model more to look at, not re-arranging what it already saw.

SYNTHESIS

A skill it was never trained on, manufactured at inference.

Two talks, one thesis. A frozen encoder holds more capability than its training objective exposes, and you unlock it with **test-time compute**, not more parameters.

RETRIEVAL TALK

recombine the geometry to buy more **relevance** on the task it was trained for.

THIS TALK

recombine the geometry to buy an **emergent new task** it was never trained for: tagging.

But test-time compute is not free lunch: it only cashes out when it feeds the model new information. Re-processing a good representation buys nothing.

WHAT IS NEW HERE

Versus the 2024-2026 training-free tagging literature.

LINE OF WORK	THEIR SETUP	OUR DELTA
RAM / RAM++	trained tagging model, curated 6.4k tag list	zero training, label space = tokenizer vocab
TagCLIP	CLIP two-tower, penultimate layer, DMAR+CWR	single omni model, last layer is the output space
PIAA	GDA whitening to close the modality gap	image/text already one space; whitening collapses it
2025 calibration (OTTER, BCA)	calibration set or target prior to de-bias	one offline background prior, zero calibration

The novelty is the combination: tokenizer vocab as the open label space, one frozen omni model for every signal, and marginal de-biasing, with all the fancy test-time machinery empirically ruled out.

AI Engineer World's Fair

A frozen model already knows more than its objective admits.

Test-time compute is how you ask.

Han Xiao · VP of AI, Elastic

@hxiao · in/hxiao87

github.com/hanxiao/jina-v5-omni-nano-test-time-image-tagging

GITHUB REPO

the code